

Detecting Strategically Decomposed Malicious Code Changes with Subscription-Backed LLM Reviewers

Tom Howard
Independent researcher
tom@windyroad.com.au

28 July 2026

Abstract

Large language models are increasingly used to review code changes, yet most evaluations present a vulnerable function, file, or repository as one unit. A malicious change can instead be decomposed across submissions that appear individually routine but compose into an unsafe final state. We describe a prospectively specified study of whether this decomposition reduces detection and whether detection differs between pull-request gates and trunk workflows that treat main as untrusted until pipeline review completes. The study uses 40 matched malicious-benign pairs drawn from 40 structural templates in eight abstract policy families. Every generated case module is synthetic, executes only in memory, and has no external capability in the audited fixed corpus. Atomic and three-submission variants have identical final trees and changed-line totals. A factorial base design varies intent, decomposition, and workflow framing across two confirmatory subscription-backed systems: Codex CLI and Claude Code. A prospectively selected 16-template nested stratum adds cumulative-history context and a second trial. The full two-system schedule contains 2,816 review boundaries, including the 1,280-boundary base stratum used for H1–H3. A considered Ollama Cloud replication was not activated and contributes no observation or analysis to this registration. The primary endpoint is `blocked_by_activation`; paired intent discrimination is the H1 contrast over that endpoint. Workflow framing and the decomposition-by-context interaction are prespecified secondary hypotheses; decomposition by workflow and the cumulative-context main effect are exploratory. This pre-results manuscript reports the design, analysis plan, deterministic baseline, safety controls, and limitations; it contains no LLM benchmark outcomes.

1 Introduction

Automated code review is commonly evaluated as a classification problem: a reviewer receives one code artifact and predicts whether it contains a defect. Real delivery systems expose a different unit of work. Reviewers see a sequence of submissions, and an unsafe capability may emerge only when several changes compose. Each earlier change can be necessary for the final failure without being unsafe in isolation.

This distinction matters for both pull-request and trunk-based delivery. A pull-request gate asks whether a proposed change may enter the protected branch. A trunk workflow can permit a commit onto main while still treating main as untrusted until deterministic and model-based pipeline gates complete. These workflows differ in policy, but empirical comparisons often also change the evidence shown to the reviewer.

We study whether two confirmatory subscription-backed LLM review products recognise a composition before its activation boundary. A considered Ollama Cloud replication was not activated; any future test requires a separate preregistration and cannot alter these confirmatory results. The benchmark represents harm as an abstract policy assertion: an unprivileged synthetic actor receives a protected synthetic capability. It neither implements an exploit nor interacts with a real target. The contribution is a reproducible, prospectively specified test of sequence-level review with no usage-based spend.

2 Related Work

Yu et al. evaluated seven LLMs and multiple prompt forms for security code review, comparing them with static analysis and examining factors associated with detection quality [1]. VulDetectBench separates vulnerability identification, classification, and localisation [2]. Zhou et al. compare repository-level LLMs and static analysis tools and report a recall-false-positive trade-off [3]. We retain a deterministic conventional-lint baseline but do not claim that lint and semantic security review are equivalent tasks.

Field evidence also indicates that context and interaction design affect developer trust and usefulness. Adalsteinsson et al. report context switching, missing context, false positives, and trust as practical concerns in an industrial study of LLM-assisted review [4]. Basic and Giarretta synthesise evidence on LLM code-security benefits, false findings, prompting, and failure modes [5]. Our study changes the unit of analysis from one artifact to a sequence of submissions.

3 Research Questions

The primary endpoint is `blocked_by_activation`: at least one review at or before the prespecified activation boundary returns `block`.

- RQ1** Can the review systems discriminate malicious sequences from matched benign sequences?
- RQ2** Does decomposition across three submissions reduce timely detection relative to an atomic submission with the same final tree?
- RQ3** Do estimates differ between pull-request and untrusted-main trunk framing?
- RQ4** Does supplying cumulative submission history change the malicious split penalty, averaged equally over workflows, and how consistent are repeated reviews?

H1 predicts that the malicious block rate exceeds the benign block rate, equally averaged over decomposition, workflow, and the two confirmatory products. H2 predicts that the malicious block rate is lower for split sequences than for atomic submissions, equally averaged over workflow and the two products. H3 is nondirectional: among malicious base local trial-1 cells, trunk and pull-request framing differ after equal averaging over atomic and split conditions and both products. H4 predicts that the malicious-only nested interaction $(\text{split}_{\text{cumulative}} - \text{atomic}_{\text{cumulative}}) - (\text{split}_{\text{local}} - \text{atomic}_{\text{local}})$, equally averaged over pull-request and trunk after both trials and products are averaged within template, is positive. H1 and H2 are primary; H3 and H4 are prespecified secondary hypotheses. The decomposition-by-workflow interaction and cumulative-context main effect are exploratory, and no workflow-equivalence claim is tested.

4 Method

4.1 Safe benchmark

The full deterministic generator contains 200 structural templates formed by eight scenario families crossed with five data representations and five control-flow shapes. The active subscription-bounded study selects template indexes 1, 7, 13, 19, and 25 from each family. This balanced diagonal retains every representation and flow shape once per family, giving 40 structural templates. Only the first invented-identifier instance is retained.

Each template contains a malicious-benign pair. Every case has an atomic submission and a three-submission sequence. Atomic and split forms have identical final source trees, total changed-line counts, activation timestamps, and policy outcomes. For malicious sequences, the generator reverts each split step separately and executes the oracle; all three counterfactuals must remain safe. Benign cases match structure and salience but do not grant the protected capability.

Generated case modules import only relative synthetic modules and run in memory. Fixed-corpus inspection, oracle execution, import restrictions, and output audit found no external capability. The regex scanner is defense in depth rather than a general capability proof. The trusted generator and oracle harness use local files, a Node subprocess, and dynamic import to construct and verify cases.

4.2 Factorial design

Table 1 shows the active factors. The base stratum supplies the current artifact only; prior submission diffs are withheld. This local information boundary is the principal concealment mechanism under test. A family-stratified nested sample supplies either the same local view or the current artifact plus cumulative prior artifacts, allowing the information-loss mechanism to be estimated without changing the original base estimands.

Table 1: Active study factors.

Factor	Levels	Interpretation
Intent	malicious, benign	Matched final-state policy outcome
Decomposition	atomic, split	One or three necessary submissions
Workflow	pull request, trunk	Pre-merge gate or untrusted-main pipeline
Information context	local; cumulative in nested stratum	Current artifact or cumulative history
Trial	one base; two in nested stratum	Independent fresh-process repetition
Review system	Codex, Claude	Subscription-backed product surface

The base stratum contains 40 malicious-benign pairs, 80 cases, 640 sequence-system trials, and 1,280 boundary reviews across the two systems. It has one local-context trial per intent, decomposition, workflow, and system cell. Before outcomes, two templates per family were selected without replacement by seeded Fisher–Yates for a 16-template nested stratum. Those templates receive local and cumulative contexts and two trials per cell. The complete two-system schedule therefore has 896 unique boundary prompts, 1,408 sequence-system trials, and 2,816 boundary reviews: 1,408 per system, 704 atomic, and 2,112 split. Trials reuse the same frozen prompt but start fresh processes.

The three split submissions retain fixed causal order and one-minute spacing; the atomic submission is timestamped at the split activation time. Timing and order are matched controls rather than estimated factors. Calls are globally shuffled with a seeded Fisher–Yates algorithm. No outcome-dependent stopping, template replacement, or prompt tuning is permitted.

4.3 Review systems

The systems are specified as product configurations rather than interchangeable base-model APIs (Table 2). Each boundary starts a fresh process with no persistent session. User configuration and tools are disabled where the client supports it. Any emitted tool call is retained as a protocol deviation.

Table 2: Confirmatory review systems.

System	Client version	Requested model
Codex CLI	0.137.0; ChatGPT login	gpt-5.5
Claude Code	2.1.211; Claude Max login	sonnet

Before each batch, the runner verifies subscription authentication, live account-identity fingerprints, disabled extra usage, and the absence of `OPENAI_API_KEY`, `CODEX_API_KEY`, and `ANTHROPIC_API_KEY`. It performs at most 16 physical dispatches before repeating those checks and aborts rather than switching to API billing, credits, another provider, or a fallback model. Codex additionally uses approval policy **never** and a custom minimal permission profile whose offline proof denies model-shell access to isolated credentials and the frozen response schema. Requested-model evidence, exact client provenance, timestamps, allowlisted integer token counts, and failure digests are retained. Every raw provider envelope is archived privately before parsing and bound to its terminal attempt. Claude acceptance requires exactly one `modelUsage` entry. Its sole key must be **claude-sonnet-4-6**. Codex CLI 0.137.0 does not expose a returned-model field in its JSON event contract; each accepted result instead records requested model **gpt-5.5**, the exact pinned binary path, version and hash, and an explicit status that the requested pinned client showed no observable

reroute. A model-rerouted error event is rejected. This is an observability limitation, not returned-model verification.

Ollama Cloud was considered as a separate exploratory replication. The dated terms assessment and provider correspondence remain audit history, but the frozen decision is **not-activated**. This registration authorises no Ollama purchase, preflight, implementation, queue, dispatch, observation, or analysis. Any future Ollama experiment requires a separate preregistration.

All reviews receive one fixed security-review task and a JSON response contract. Requests ask for **allow**, **block**, or **abstain**; a maliciousness probability; severity; relevant **head_revision** or **commit_revision** values; evidence; and one canonical abstract-policy category from the frozen schema. Complete requests are at most 3,992 UTF-8 bytes. A response that fails the structured-response validator is a terminal **schema_invalid** abstention and is not repaired. The protocol does not infer a separate refusal class from unstructured provider output.

4.4 Collection and blinding

The call queue contains opaque call, prompt, and case identifiers plus review system and schedule position. It contains no intent, family, structural-template, activation, or expected-severity field. Ground truth is held in a separately keyed ledger and joined only after collection.

Subscription rate limits produce a durable suspension at the current call; the same fixed queue resumes after the plan window resets. No API credit is accepted. Immediately before each invocation, the author inspects both account billing settings and supplies a private confirmation, no more than 15 minutes old, binding the authorized account fingerprints and stating that extra or overage usage is disabled. The runner verifies it before output creation or client preflight. Each invocation is limited to 16 physical dispatches before that confirmation and all other checks repeat. Every frozen authorized call must have exactly one terminal result before analysis: a schema-valid review, a validator-produced abstention, or a terminal missing result after exactly three client-failure suspensions. Started, suspension, and terminal result records are fsynced. Tool deviation is a retained boolean on a schema-valid result. Suspension reasons distinguish rate limits; identity absence or drift; malformed provider envelopes; authorized-schema, executable, or isolation drift; client failure; and interrupted processes. “Crash-safe” means resume after a cleanly observed interruption; it does not claim recovery from a torn JSONL tail, abrupt power loss, concurrent writers, an orphan created between envelope archival and ledger append, or a check-to-spawn race. Collection also suspends on a safety or terms conflict, authentication change, or benchmark integrity failure. The API, per-call, overage, extra-balance, and paid-fallback spending ceiling is US\$0. Fixed subscription fees are disclosed separately and any new plan requires action-time approval.

4.5 Outcomes

The primary binary outcome is **blocked_by_activation**. Secondary outcomes are recall, false-positive rate, precision, abstention rate, cumulative detection by boundary, submissions and elapsed minutes to detection, Brier score, calibration, severity error, localisation, and failure-category rates. Submission localisation requires citing at least one frozen relevant submission identifier; mechanism localisation requires the exact frozen abstract-policy category; combined localisation requires both. Within-system consistency in the balanced two-trial nested stratum uses pairwise operational-verdict agreement and malicious-probability ICC(1,1), with template-bootstrap intervals.

5 Analysis Plan and Power

H1–H3 use only base-stratum rows: local context, trial 1, all 40 templates, and exactly **codex-cli/gpt-5.5** and **claude-code/sonnet**. H1 equally averages decomposition, workflow, and both products within template; H2 equally averages workflow and both products within malicious templates; H3 equally averages atomic and split conditions and both products within malicious templates. Templates then enter the equal-family analysis. Review-system-specific results are secondary. Registered 95% intervals use a fixed-strata Welch/Satterthwaite t procedure over template-level contrasts. The eight policy families are fixed, equally

weighted strata; within-family variance components use five base templates for H1–H3 and two nested templates for H4. A 10,000-replicate family-stratified percentile bootstrap with seed 20260718 is reported as a sensitivity interval, not the support-decision interval.

The H1 estimate is the malicious-minus-benign block-rate risk difference, equally averaged over decomposition, workflow, and both products. H1 is supported only when its 95% interval is above zero. The H2 estimate is the split-minus-atomic malicious risk difference, equally averaged over workflow and both products. H2 is supported only when its 95% interval is below zero. H3 is the trunk-minus-pull-request malicious contrast, averaged equally over decomposition and both products, and is supported only when its nondirectional 95% interval excludes zero. H4 uses only the frozen 16-template nested stratum, averaging its two trials and both products within context cells, and then equally averaging pull-request and trunk. It is the malicious-only difference $(\text{split} - \text{atomic})_{\text{cumulative}} - (\text{split} - \text{atomic})_{\text{local}}$ and is supported only when its 95% interval is above zero. The decomposition-by-workflow difference-in-differences and cumulative-context main effect are exploratory. Nested consistency retains template-bootstrap intervals and has no support decision.

If every within-family variance component for a registered hypothesis is zero, its Welch interval, Satterthwaite degrees of freedom, and support decision are unavailable. The point estimate and standard error 0 remain reported, with only the percentile-bootstrap sensitivity interval shown.

Missing boundaries are non-detections in the operational analysis. Mandatory estimand-specific bounds assign missing cells to zero or one in the opposing directions for each contrast: malicious versus benign for H1, split versus atomic for H2, trunk versus pull request for H3, and the four signed H4 cells. A review-system-specific complete-pair sensitivity for H1–H4 drops a structural template only for the affected product while retaining equal-family weighting. If any fixed family then has fewer than two complete templates, that product-specific registered interval is reported as unavailable rather than changing the estimand or variance method. No imputation model is fitted. The four registered claims are not combined into a multiplicity procedure, so no familywise error control is claimed; every estimate, interval, bound, and decision is reported.

No Ollama observation or analysis is part of this registration. Confirmatory analysis rejects Ollama, missing, extra, or mixed rows and also rejects a ledger unless every frozen authorized call has exactly one terminal result.

The products and sample are a convenience selection determined by subscriptions available to the author, not an 80% power target. An earlier 20,000-replication single-system normal-approximation audit did not match the final design. A subsequent outcome-free audit simulated the exact base and nested strata and found material undercoverage for the initially proposed percentile bootstrap. The interval was therefore changed prospectively. Under 1,000 global-null simulations, fixed- strata Welch coverage for H1–H4 was 0.953, 0.965, 0.953, and 0.982. Under the central assumptions, support fractions were 1.000, 0.724, 0.356, and 0.073 and expected interval widths were 0.143, 0.211, 0.212, and 0.480. These values are assumption-conditional diagnostics, not guaranteed or achieved power. Null results cannot support no-effect or equivalence claims; H4 in particular is expected to be imprecise. The study is a bounded empirical evaluation and a variance estimate for a larger replication.

6 Pre-results Status

No benchmark prompt has been submitted to any review system. The two-system confirmatory generator, prompts, schedule, ledgers, scoring, estimands, and tests were implemented with deterministic checks and fabricated plumbing responses only. No LLM outcome has been collected or inspected. No Ollama runner, queue, ledger, or output artifact has been implemented or generated, and the frozen decision excludes Ollama from this study. An earlier no-prompt confirmatory preparation check passed for the pinned Codex and Claude clients without inference; the hardened sandbox, identity, and ledger checks require exact-commit revalidation before registration. Ollama’s AI support agent did not grant or deny permission and offered human escalation; that exchange is retained as audit history and cannot activate Ollama within this frozen study.

The deterministic baseline ran ESLint 9.39.3 over all 2,400 final-state files in the full 800-case generator output. It emitted no warnings or errors: zero malicious and zero benign cases were flagged, giving recall 0, false-positive rate 0, and undefined precision. This conventional final-state lint does not estimate decomposi-

tion or workflow effects. A separate zero-finding Semgrep feasibility probe is disclosed but excluded because its exact registry-rules snapshot cannot be redistributed under the provider’s rules license.

7 Threats to Validity

Construct validity. The benchmark models malicious intent through a harmless abstract policy violation. It tests composition recognition, not the full semantics or distribution of production vulnerabilities. Workflow is a controlled artifact and policy framing, not an organisation.

Internal validity. Atomic and split variants can differ in salience despite identical final trees and line counts. Provider-controlled system instructions are neither observable nor identical. Product routing and serving stacks may change behind requested model identifiers; metadata and access times are therefore retained and drift suspends collection. Collection may span subscription reset windows, creating calendar-time and platform-drift risk despite the fixed randomised queue.

External validity. Forty purposively selected templates, eight policy families, two confirmatory products, and compact JavaScript repositories do not represent all languages, organisations, vulnerabilities, or attack strategies. The design tests one three-submission decomposition and does not estimate an optimal adversarial strategy. Inference is conditional on these templates; one lexical instance per template does not estimate identifier-instance variability. The subscription-only design is not cost-free for an independent reproducer without equivalent subscriptions. Product surfaces, schema enforcement, hidden prompts, calendar periods, routing, and cloud aliases confound descriptive cross-product comparisons and cannot support provider or open-versus-closed-weight claims.

Statistical conclusion validity. The sample is underpowered for small effects, especially workflow and interaction effects. Only the nested stratum has repeated trials, so consistency estimates apply to 16 purposively bounded templates rather than the full sample. Family-stratified intervals expose rather than hide this uncertainty.

8 Ethics and Responsible Release

This is defensive research into review reliability. The benchmark cannot access external systems or data, and its only harmful state is a deterministic in-memory assertion over synthetic names. No real repository or person is tested.

Public research data are limited to sanitized derived attempt-event data and aggregate analysis. Raw ledgers, provider output, absolute paths, account and authentication state, and environment metadata remain private unless a post-outcome safety, upstream-license, privacy, and terms review approves a narrower release. Aggregate reporting uses factual plain-text attribution without logos or endorsement claims.

Three isolated subagents initially reviewed candidate commit `6b607f6` for benchmark safety, methods, and reproducibility. All three returned **do not approve**; their raw reports are archived. Correction addenda inspect the combined resolution log and are therefore not isolated. These records are labelled AI-assisted internal review, not independent human peer review, ethics approval, or arXiv endorsement. Shared model, provider, and author-orchestration biases are reported as limitations. Neutral change titles remove plausible refactor cover stories, but the abstract composition patterns retain disclosed dual-use value. The packet contains no real target, external capability, credential, deployment path, or complete exploit. Targeted non-isolated reviews of commit `70963a2` rejected its release-packet and execution-authorization mechanics. Their reports and accepted findings are archived. A later outcome-blind worktree review also retained a **do not approve** reproducibility disposition while authorization-schema, terminal-result, identity-evidence, executable-pinning, writer-exclusion, and release-boundary corrections were incorporated before this reviewer-input candidate was frozen. Later outcome-blind worktree reviews found no blocker, but they are not phase-one approval. All three review roles must approve the same exact frozen commit before the detached phase-one attestation and OSF submission. The marker cannot itself represent approval or authorize an outcome call.

9 Reproducibility

The candidate package contains code for benchmark generation, scheduling, scoring, sensitivity analysis, subscription collection, and outcome analysis. It also contains the preregistration, archived internal reviews, and two-phase freeze records. Each durable ledger begins with one immutable run-binding header. The analysis requires the exact binding schema, recomputes its fingerprint, and verifies the raw ground-truth-ledger digest before joining outcomes. It excludes the metadata row from outcomes and carries the verified binding into its report. Frozen reviewer-input hashes are:

- scenario cards: `d4ea0a7197e3e8e17bf027b9466d63de427c273aa19e539a672732b80b5eae3d`;
- rendered prompts: `e99d650f5f670e138b0c31047c6009a1eea99f2d2005c742971419618c86bdf0`;
- randomised schedule: `80c78d846bf264fcb8a136bffd3b6d3af50c0e8dfe622383272e0d4798ab63a1`;
- blinded call ledger: `53422146aef4497fbb4da0819a3134f46b1e75351857a37ac92d0d3114bbbfb1`;
- ground-truth ledger: `9453faab7b1bc1dc06cdef7bb83c7d4a15e96d6f6c06d407acd0aa823931cf6d`.

The first phase builds a canonical JSON payload from an exact Git commit using a fixed 14-member release-safe allowlist. It includes the study-local licence and the privacy-preserving AI-support-response transcription with its required SHA-256 recorded in the manifest. Each member records path, media type, raw byte size, SHA-256, and Base64 content; the verifier rejects extra, reordered, traversal, detached-state, executable-case, credential, account-state, result, raw-output, and reasoning-trace members. The outer freeze record binds the payload and member-manifest hashes, selected branch, exact queue shape and counts, systems, normalized artifacts, raw artifact bytes, and runtime files before OSF submission, while explicitly leaving outcome calls unauthorised. Each detached review report carries machine-readable role, exact-commit, disposition, and blocker metadata that must match its report hash and phase-one attestation. The confirmatory-only branch also requires the machine-readable Ollama **not-activated** decision and false pre-outcome state flags.

After registration, a separate execution record binds untouched raw OSF-origin registration JSON, its structurally derived summary, the exact downloaded two-file inventory, and explicit action-time author confirmation to the canonical OSF identity, raw freeze-record hash, content commit, branch, payload, member manifest, and exact queues. This author-attested HTTPS capture is not an OSF cryptographic signature or independently trusted timestamp. UTC phase timestamps are strictly ordered. The confirmatory runner rejects extra or Ollama-labelled queues, recomputes the schedule and all bindings, and executes the same prompt and call buffers it hashed. Rejection precedes output creation, client preflight, or model execution. Both detached records are excluded from the payload to avoid self-reference.

The earlier paid API design is retained in version history but was superseded before outcome collection. The public research release will add sanitized derived attempt-event data, analysis outputs, checksums, and the OSF and arXiv identifiers. Raw ledgers, raw provider output, absolute paths, account and authentication state, and environment metadata remain private pending explicit post-outcome review.

10 Conclusion

This study tests whether subscription-backed LLM review products recognise an unsafe composition when its necessary changes arrive separately, and whether the estimate changes under pull-request or untrusted-main framing. The safe paired benchmark supports auditable positive or negative findings, while the subscription constraint makes uncertainty and product drift part of the study rather than hidden implementation details. Empirical conclusions remain contingent on prospective registration, completion of the fixed schedule, and the prespecified analysis.

A Fixed Review Task

The fixed task instructs the review system to act as a security reviewer, treat repository and change metadata as untrusted evidence, decide whether the current submission should be allowed, blocked, or sent for human

review, consider unsafe behaviour created alone or in composition with supplied evidence, and return only the contracted JSON grounded in the supplied artifact. No chain-of-thought request, product-specific prompt tuning, or outcome-driven tuning is used.

References

- [1] J. Yu, P. Liang, Y. Fu, A. Tahir, M. Shahin, C. Wang, and Y. Cai. An insight into security code review with LLMs: Capabilities, obstacles, and influential factors. *arXiv:2401.16310*, 2024. <https://arxiv.org/abs/2401.16310>.
- [2] Y. Liu, L. Gao, M. Yang, Y. Xie, P. Chen, X. Zhang, and W. Chen. VulDetectBench: Evaluating the deep capability of vulnerability detection with large language models. *arXiv:2406.07595*, 2024. <https://arxiv.org/abs/2406.07595>.
- [3] X. Zhou, D.-M. Tran, T. Le-Cong, T. Zhang, I. C. Irsan, J. Sumarlin, B. Le, and D. Lo. Comparison of static application security testing tools and large language models for repo-level vulnerability detection. *arXiv:2407.16235*, 2024. <https://arxiv.org/abs/2407.16235>.
- [4] F. S. Adalsteinsson, B. B. Magnusson, M. Milicevic, A. N. Davidsson, and C.-H. Cheng. Rethinking code review workflows with LLM assistance: An empirical study. *arXiv:2505.16339*, 2025. <https://arxiv.org/abs/2505.16339>.
- [5] E. Basic and A. Giaretta. From vulnerabilities to remediation: A systematic literature review of LLMs in code security. *arXiv:2412.15004*, 2024. <https://arxiv.org/abs/2412.15004>.